Dynamic Covariates: Predicting density and assessing factors driving marbled murrelet distribution in Puget Sound using hierarchical distance sampling

Code by S.J. Gillman

Description & Sources

1. **SalishSeaCast Model Outputs**

- SalishSeaCast Descriptions for each variable can be found in their background/metadata. All files were downloaded from the SalishSeaCast ERDDAP site.
- These data included outputs from their chemistry, biology, physics, and turbidity fields. Further information for each are provided below.

2. **Maxent Habitat Rasters**

- Raster files provided by from the Northwest Forest Plan's 30-year Marbled Murrelet nesting habitat assessment (Duarte et al 2025; in review).

Required Packages

```
library(sf)
library(terra)
library(tidyverse)
library(ggplot2)
library(lubridate)
library(gstat)
```

Required Files Previously Made or Downloaded

- `puget_poly.gpkg`
- `reduced_area.gpkg`
- `SS_Bathymetry.tiff` (downloaded)
- `grid2km_hex.gpkg`
- `all_tracks_2026_final.gpkg`

```
puget <- st_read("../GIS_COVS/puget_poly.gpkg")

ss_bath <- terra::rast("../GIS_COVS/SS_Bathymetry/SS_Bathymetry.tif")
studyarea <- st_read("../GIS_COVS/reduced_area.gpkg")

hex_grids <- st_read("GIS_COVS/grid2km_hex.gpkg")
tracklines <- st_read("../GIS_COVS/all_tracklines_2026_final.gpkg")

track_buffer <- tracklines %>%
  mutate(numid = row_number()) %>%
  st_buffer(dist = 215)
```

SalishSeaCast

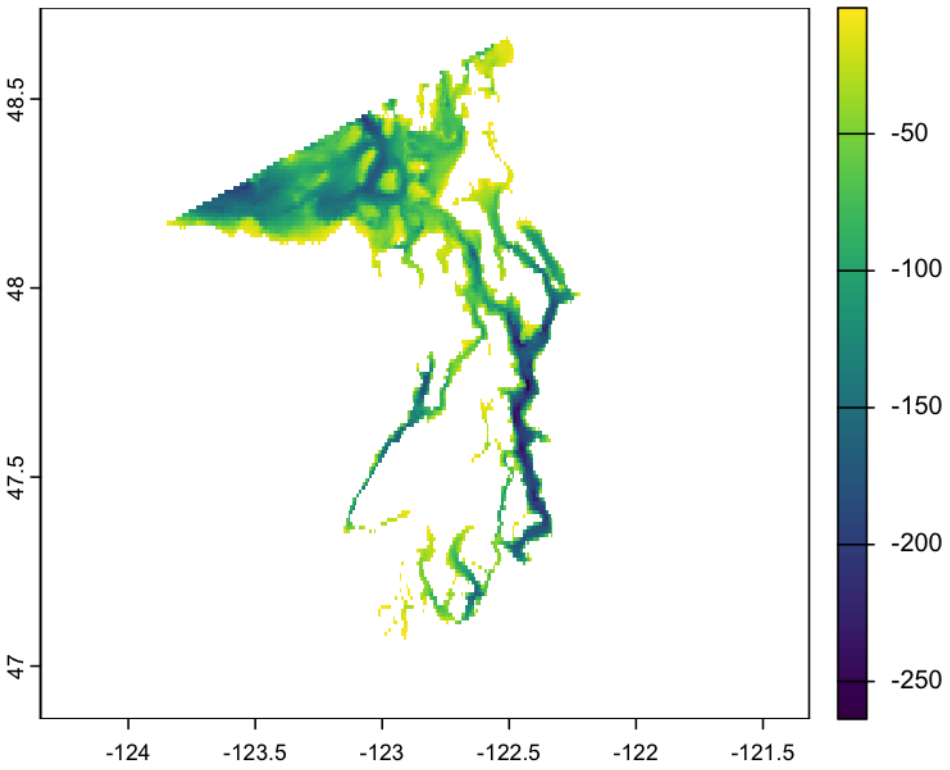
SalishSeaCast Descriptions for each variable can be found in their background/metadata. All files were downloaded from the SalishSeaCast ERDDAP site

The `ubcSSnBathymetry.csv` is the grid SalishSeaCast geo-location and bathymetry for the model they create and used here to link specific monthly observations to the study grids they are located in. I do not use their bathymetry data and only use the information for geo-locations.

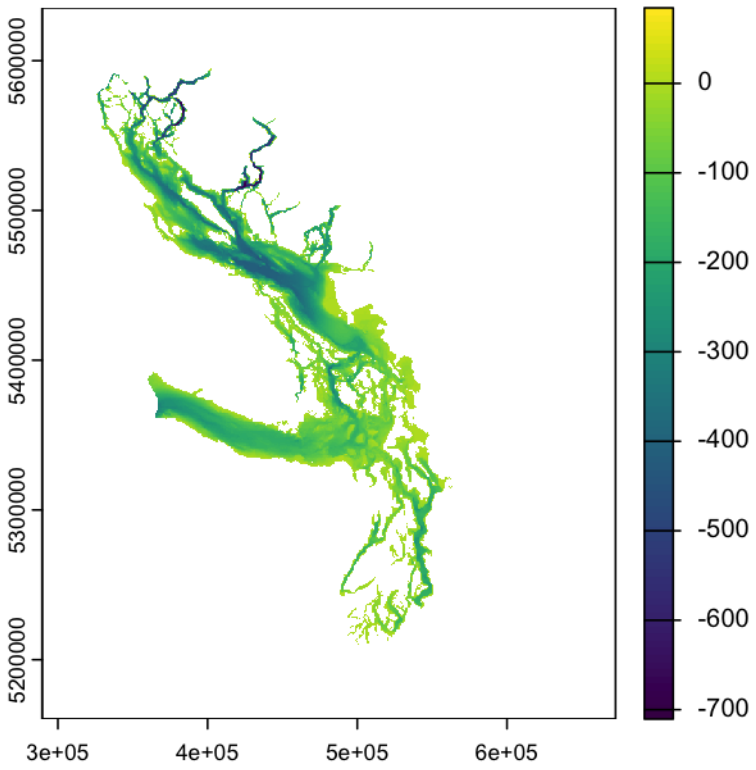
The SalishSeaCast NEMO Model Grid was downloaded from the SalishSeaCast NEMO model (Soontiens et al, 2016; Soontiens and Allen, 2017) their ERDDAP server (https://salishsea.eos.ubc.ca/erddap/) on **01/20/2025** for all months from 2012 to end of 2024.

Setting up grid

```
### setting up seacast info get gridxy to lat long
nemobath <- read.csv("../GIS_COVS/ubcSSnBathymetry.csv")

## get rid of the top row
nemobath <- nemobath %>%
 filter(gridY != "count") %>%
  mutate(gridxy = paste0(gridX,"-",gridY),
         longitude= as.numeric(longitude),
         latitude = as.numeric(latitude),
        gridX= as.numeric(gridX),
        gridY= as.numeric(gridY),
        bathymetry =-1*as.numeric(bathymetry))


# Convert to a SpatVector (terra object)
points_vect <- vect(nemobath, geom = c("longitude", "latitude"),
                    crs = "EPSG:4326")

r <- rast(extent = ext(points_vect),
  crs = "EPSG:4326")


bathymetry_raster <- rasterize(points_vect, r, field = "bathymetry",
                               fun = "mean")


plot(bathymetry_raster)
```




```
plot(ss_bath)
```



Chemistry Fields

3d SKOG carbon chemistry and oxygen fields from the SalishSeaCast NEMO model (Soontiens et al, 2016; Soontiens and Allen, 2017, JarnÃkovÃ¡ et al, 2022) were downloaded from their ERDDAP server on **01/20/2025** from dataset ubcSSg3DChemistryFields1moV21-11 for all months from 2012 to end of 2024.

Dissolved Inorganic Carbon not included in the model

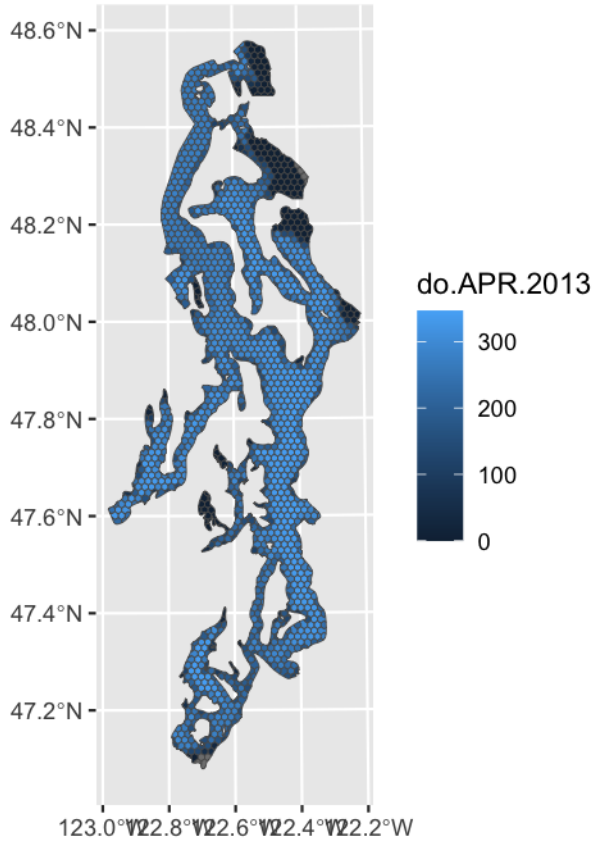
```
folder_path <- "../GIS_COVS/SSC_CHM"

csv_files <- list.files(folder_path, pattern = "\\.csv$", full.names = TRUE)

# Generate simplified names like dfbio1, dfbio2, etc.
simplified_names <- paste0("dfchm", seq_along(csv_files))

# Load all CSV files into a list with simplified names
csv_list <- lapply(csv_files, read.csv)
names(csv_list) <- simplified_names

dic_list<- list()
for (i in 1:length(csv_list)){
 dic_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
```

```
    dplyr::select(c(time, gridxy,dissolved_inorganic_carbon)) %>%
    mutate(dissolved_inorganic_carbon = as.numeric(dissolved_inorganic_carbon),
           year = lubridate::year(lubridate::ymd_hms(time)),
           mon = toupper(format(lubridate::ymd_hms(time), "%b")),
           monyear = paste0(mon, "-", year)) %>%
   mutate(dissolved_inorganic_carbon= ifelse(is.nan(dissolved_inorganic_carbon),
                                             NA,
                                             dissolved_inorganic_carbon)) %>%
    dplyr::select(gridxy,monyear, dissolved_inorganic_carbon) %>%
    pivot_wider(names_from = monyear,
      values_from = dissolved_inorganic_carbon,
      names_prefix = "dic-")
}

combined_csvdic <- bind_rows(dic_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_dic <- vect(combined_csvdic, geom = c("longitude", "latitude"),
                   crs = "EPSG:4326")

columns_to_rasterize <- setdiff(names(combined_csvdic),
                                c("latitude", "longitude"))

dic_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_dic),
    crs = "EPSG:4326")

  # Rasterize the points for the current column
  raster_layer <- rasterize(points_dic, r, field = col, fun = "mean", na.rm=T)

  # Add the raster to the list
  dic_raster[[col]] <- raster_layer
}


# Combine all rasters into a stack
dic_stack <- rast(dic_raster)

puget_transformed <- st_transform(puget, crs= crs(dic_stack))

cropped_raster <- crop(dic_stack, puget_transformed)
#mask
masked_raster <- terra::mask(cropped_raster, puget_transformed)

hex_grid_transformed <- st_transform(hex_grids, crs= crs(dic_stack))
```

```r
track_transformed <- st_transform(track_buffer,crs= crs(dic_stack))

dic_ext <- terra::extract(masked_raster,hex_grid_transformed,
                          fun = "mean",weights= T,ID=T,na.rm=T)


hex_dic <- hex_grids %>%
  full_join(dic_ext %>%
              rename(numid= ID), by = "numid")


dic_ext_track <- terra::extract(masked_raster, track_transformed,
                                fun = "mean",weights= T,ID=T,na.rm=T)


track_dic <- track_buffer %>%
  full_join(dic_ext_track %>%
              rename(numid= ID), by = "numid")
```

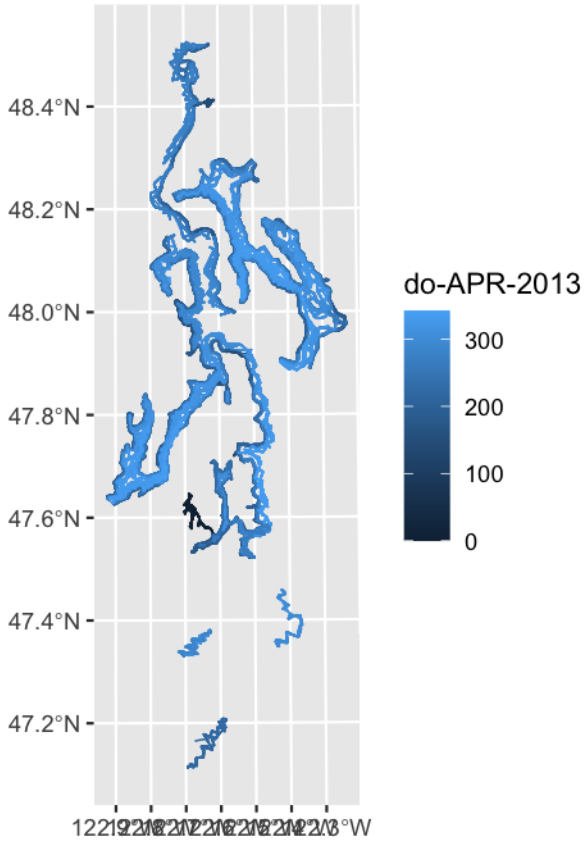
```r
do_list<- list()
for (i in 1:length(csv_list)){
  do_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
    dplyr::select(c(time, gridxy,dissolved_oxygen)) %>%
    mutate(dissolved_oxygen = as.numeric(dissolved_oxygen),
           year = lubridate::year(lubridate::ymd_hms(time)),
           mon = toupper(format(lubridate::ymd_hms(time), "%b")),
           monyear = paste0(mon, "-", year)) %>%
    mutate(dissolved_oxygen = ifelse(is.nan(dissolved_oxygen),NA,
                                     dissolved_oxygen)) %>%
    dplyr::select(gridxy,monyear, dissolved_oxygen) %>%
    pivot_wider(names_from = monyear,
                values_from = dissolved_oxygen,
                names_prefix = "do-")
}

combined_csvdo <- bind_rows(do_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_do <- vect(combined_csvdo, geom = c("longitude", "latitude"),
                  crs = "EPSG:4326")

columns_to_rasterize <- setdiff(names(combined_csvdo),
                                c("latitude", "longitude"))

do_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
```

```
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_do),
    crs = "EPSG:4326")

  # Rasterize the points for the current column
  raster_layer <- rasterize(points_do, r, field = col,
                            fun = "mean", na.rm=T)

  # Add the raster to the list
  do_raster[[col]] <- raster_layer
}

# Combine all rasters into a stack
do_stack <- rast(do_raster)

puget_transformed <- st_transform(puget, crs= crs(do_stack))

cropped_raster <- crop(do_stack, puget_transformed)
#mask
masked_raster <- terra::mask(cropped_raster, puget_transformed)

hex_grid_transformed <- st_transform(hex_grids, crs= crs(do_stack))

track_transformed <- st_transform(track_buffer, crs= crs(do_stack))

do_ext <- terra::extract(masked_raster,hex_grid_transformed,
                         fun = "mean",weights= T,ID=T,na.rm=T)

hex_chem <- hex_dic %>%
  full_join(do_ext %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = hex_chem, aes(fill=`do-APR-2013`))
```



Dissolved Oxygen

```
do_ext_track <- terra::extract(masked_raster,track_transformed,
                               fun = "mean",weights= T,ID=T,na.rm=T)

track_chem <- track_dic %>%
  full_join(do_ext_track %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = track_chem, aes(fill=`do-APR-2013`))
```



SAVE INTERMEDIATE

```
st_write(hex_chem, "GIS_COVS/hex_chem.gpkg", delete_layer = TRUE)
st_write(track_chem, "GIS_COVS/track_chem.gpkg", delete_layer = TRUE)

hex_chem_df <- hex_chem %>%
  st_drop_geometry()

track_chem_df <- track_chem %>%
  st_drop_geometry()

write.csv(track_chem_df, "GIS_COVS/track_chem.csv")
write.csv(hex_chem_df, "GIS_COVS/hex_chem.csv")
```

Cleaning up space

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea","ss_bath","hex_chem",
                          "track_chem","nemobath")))
```

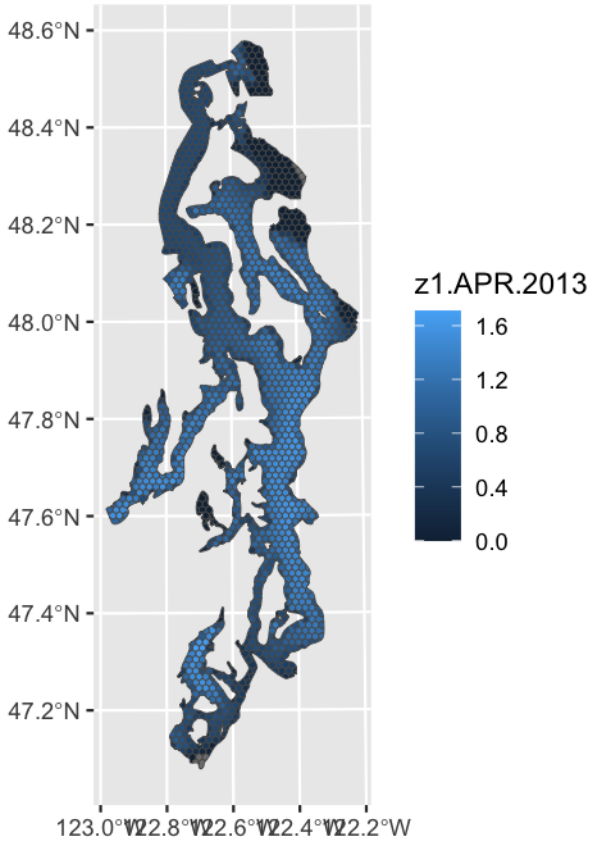
Biology Fields

3d SMELT biological model field values from the SalishSeaCast NEMO model (Soontiens et al, 2016; Moore-Maley et al, 2016; Soontiens and Allen, 2017; Olson et al, 2020; Suchy et al, 2023) were downloaded from their ERDDAP server on **01/20/2025** from dataset ubcSSg3DBiologyFields1moV21-11 for all months from 2012 to end of 2024.

- Z2_zooplankton
- Z1_zooplankton
- Diatoms

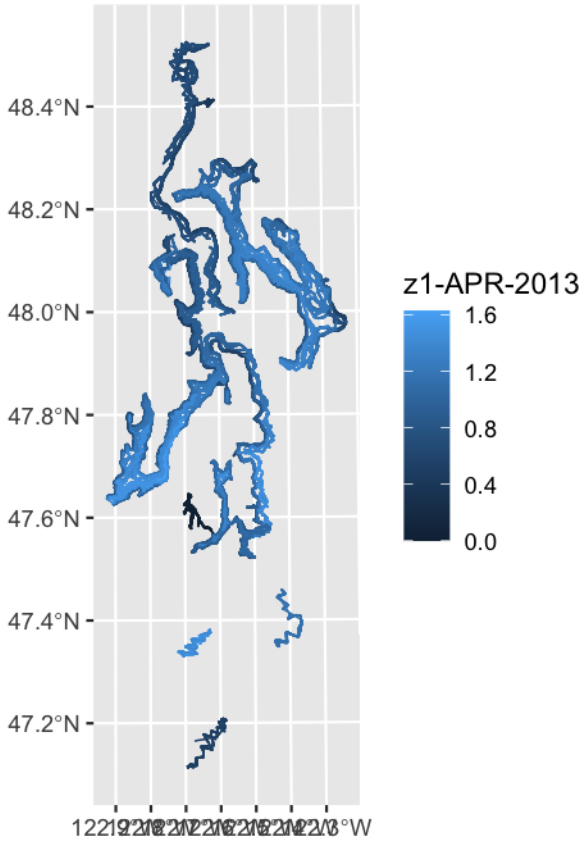
```
folder_path <- "../GIS_COVS/SSC_BIO"

csv_files <- list.files(folder_path, pattern = "\\.csv$", full.names = TRUE)

# Generate simplified names like dfbio1, dfbio2, etc.
simplified_names <- paste0("dfbio", seq_along(csv_files))

# Load all CSV files into a list with simplified names
csv_list <- lapply(csv_files, read.csv)
names(csv_list) <- simplified_names

z1_list<- list()
for (i in 1:length(csv_list)){
z1_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
    dplyr::select(c(time, gridxy,z1_zooplankton)) %>%
    mutate(z1_zooplankton = as.numeric(z1_zooplankton),
      year = lubridate::year(lubridate::ymd_hms(time)),
      mon = toupper(format(lubridate::ymd_hms(time), "%b")),
      monyear = paste0(mon, "-", year)) %>%
   mutate(z1_zooplankton= ifelse(is.nan(z1_zooplankton),NA,
                                 z1_zooplankton)) %>%
    dplyr::select(gridxy,monyear, z1_zooplankton) %>%
    pivot_wider(names_from = monyear,
      values_from = z1_zooplankton,
      names_prefix = "z1-")
}

combined_csvz1 <- bind_rows(z1_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_z1 <- vect(combined_csvz1, geom = c("longitude", "latitude"),
                  crs = "EPSG:4326")

columns_to_rasterize <- setdiff(names(combined_csvz1),
                                c("latitude", "longitude"))

z1_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_z1),
    crs = "EPSG:4326")
```

```
  # Rasterize the points for the current column
  raster_layer <- rasterize(points_z1, r, field = col,
                            fun = "mean", na.rm=T)

  # Add the raster to the list
  z1_raster[[col]] <- raster_layer
}


# Combine all rasters into a stack
z1_stack <- rast(z1_raster)

puget_transformed <- st_transform(puget, crs= crs(z1_stack))

cropped_raster <- crop(z1_stack, puget_transformed)
#mask

masked_raster <- terra::mask(cropped_raster, puget_transformed)

hex_grid_transformed <- st_transform(hex_grids, crs= crs(z1_stack))

z1_ext <- terra::extract(masked_raster,hex_grid_transformed,
                         fun = "mean",weights= T,ID=T,na.rm=T)


hex_z1 <- hex_grids %>%
  full_join(z1_ext %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = hex_z1, aes(fill=`z1-APR-2013`))
```



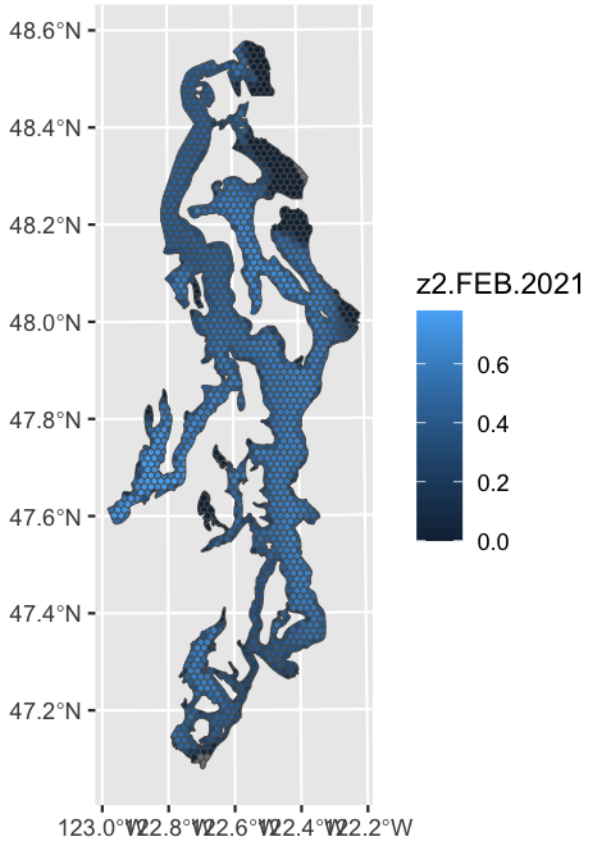
Z1 Zooplankton

```
track_transformed <- st_transform(track_buffer, crs= crs(z1_stack))

z1_ext_track <- terra::extract(masked_raster,track_transformed,
                               fun = "mean",weights= T,ID=T,na.rm=T)


track_z1 <- track_buffer %>%
  full_join(z1_ext_track %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = track_z1, aes(fill=`z1-APR-2013`))
```



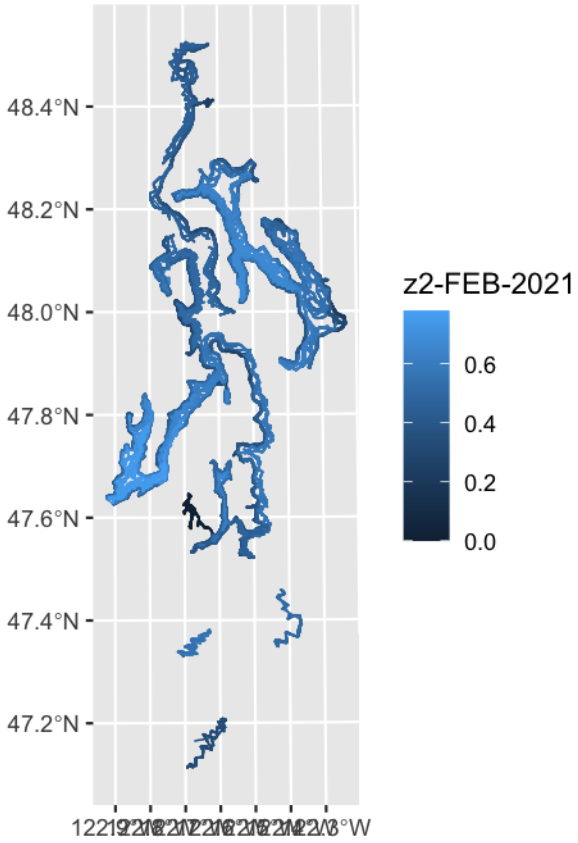

```
z2_list<- list()
for (i in 1:length(csv_list)){
  z2_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
    dplyr::select(c(time, gridxy,z2_zooplankton)) %>%
    mutate(z2_zooplankton = as.numeric(z2_zooplankton),
           year = lubridate::year(lubridate::ymd_hms(time)),
           mon = toupper(format(lubridate::ymd_hms(time), "%b")),
           monyear = paste0(mon, "-", year)) %>%
    mutate(z2_zooplankton = ifelse(is.nan(z2_zooplankton), NA,
                                   z2_zooplankton)) %>%
    dplyr::select(gridxy,monyear, z2_zooplankton) %>%
    pivot_wider(names_from = monyear,
                values_from = z2_zooplankton,
                names_prefix = "z2-")
}

combined_csvz2 <- bind_rows(z2_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_z2 <- vect(combined_csvz2, geom = c("longitude", "latitude"),
```

```
                  crs = "EPSG:4326")

columns_to_rasterize <- setdiff(names(combined_csvz2),
                                c("latitude", "longitude"))

z2_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_z2),
    crs = "EPSG:4326")

  # Rasterize the points for the current column
  raster_layer <- rasterize(points_z2, r, field = col, fun = "mean", na.rm=T)

  # Add the raster to the list
  z2_raster[[col]] <- raster_layer
}


# Combine all rasters into a stack
z2_stack <- rast(z2_raster)

puget_transformed <- st_transform(puget, crs= crs(z2_stack))

cropped_raster <- crop(z2_stack, puget_transformed)
#mask
masked_raster <- terra::mask(cropped_raster, puget_transformed)

hex_grid_transformed <- st_transform(hex_grids, crs= crs(z2_stack))

z2_ext <- terra::extract(masked_raster,hex_grid_transformed,
                         fun = "mean",weights= T,ID=T,na.rm=T)

hex_z2 <- hex_z1 %>%
  full_join(z2_ext %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = hex_z2, aes(fill=`z2-FEB-2021`))
```



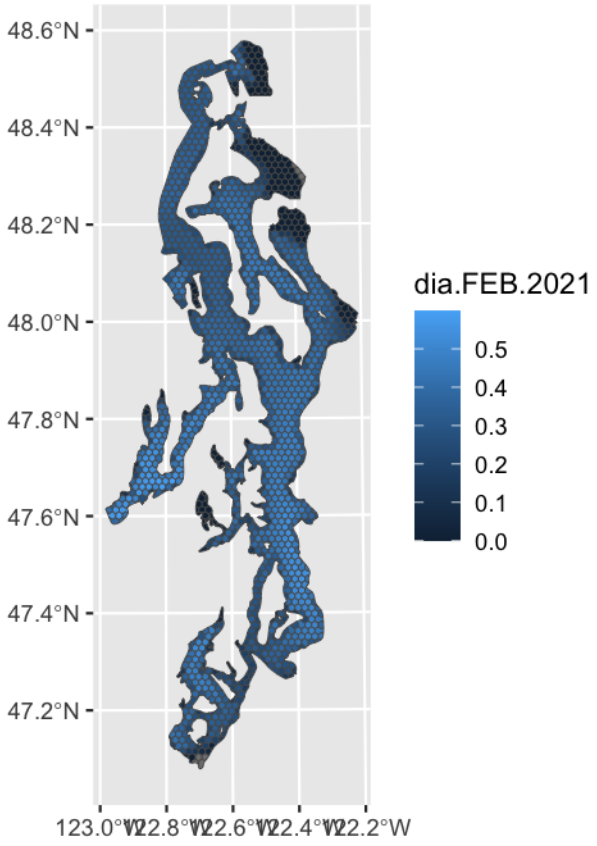
Z2 Zooplankton

```
track_transformed <- st_transform(track_buffer, crs= crs(z2_stack))

z2_ext_track <- terra::extract(masked_raster,track_transformed,
                               fun = "mean",weights= T,ID=T,na.rm=T)

track_z2 <- track_z1 %>%
  full_join(z2_ext_track %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = track_z2, aes(fill=`z2-FEB-2021`))
```



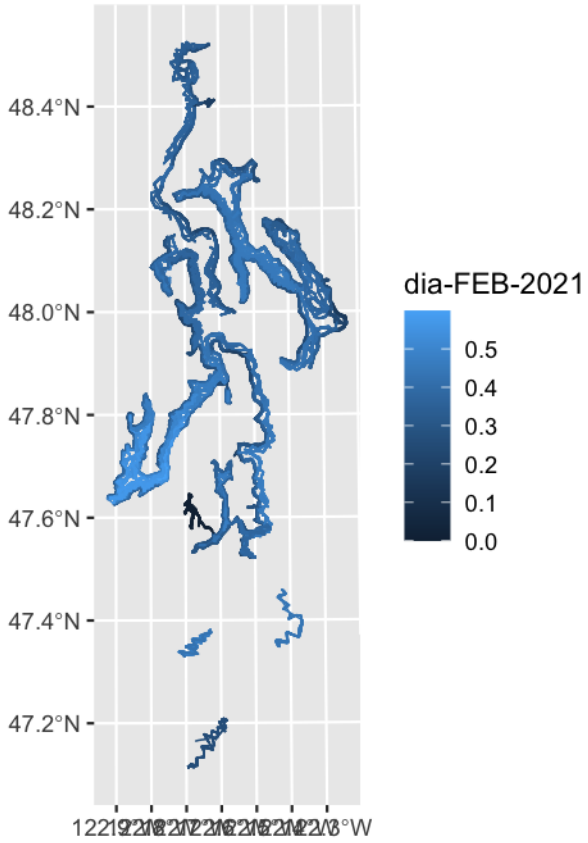

```
dia_list<- list()
for (i in 1:length(csv_list)){
  dia_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
    dplyr::select(c(time, gridxy,diatoms)) %>%
    mutate(diatoms = as.numeric(diatoms),
           year = lubridate::year(lubridate::ymd_hms(time)),
           mon = toupper(format(lubridate::ymd_hms(time), "%b")),
           monyear = paste0(mon, "-", year)) %>%
    mutate(diatoms = ifelse(is.nan(diatoms), NA,diatoms)) %>%
    dplyr::select(gridxy,monyear, diatoms) %>%
    pivot_wider(names_from = monyear,
                values_from = diatoms,
                names_prefix = "dia-")
}

combined_csvdia <- bind_rows(dia_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_dia <- vect(combined_csvdia, geom = c("longitude", "latitude"),
                   crs = "EPSG:4326")
```

```r
columns_to_rasterize <- setdiff(names(combined_csvdia),
                                c("latitude", "longitude"))

dia_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_dia),
            crs = "EPSG:4326")

  # Rasterize the points for the current column
  raster_layer <- rasterize(points_dia, r, field = col, fun = "mean", na.rm=T)

  # Add the raster to the list
  dia_raster[[col]] <- raster_layer
}


# Combine all rasters into a stack
dia_stack <- rast(dia_raster)

puget_transformed <- st_transform(puget, crs= crs(dia_stack))

cropped_raster <- crop(dia_stack, puget_transformed)
#mask
masked_raster <- terra::mask(cropped_raster, puget_transformed)


hex_grid_transformed <- st_transform(hex_grids, crs= crs(dia_stack))

dia_ext <- terra::extract(masked_raster,hex_grid_transformed,
                          fun = "mean",weights= T,ID=T,na.rm=T)

hex_bio <- hex_z2 %>%
  full_join(dia_ext %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = hex_bio, aes(fill=`dia-FEB-2021`))
```



Diatoms

```
track_transformed <- st_transform(track_buffer, crs= crs(dia_stack))

dia_ext_track <- terra::extract(masked_raster,track_transformed,
                                fun = "mean",weights= T,ID=T,na.rm=T)

track_bio <- track_z2 %>%
  full_join(dia_ext_track %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = track_bio, aes(fill=`dia-FEB-2021`))
```



SAVE INTERMEDIATE

```
st_write(hex_bio, "GIS_COVS/hex_bio.gpkg", delete_layer = TRUE)
st_write(track_bio, "GIS_COVS/track_bio.gpkg", delete_layer = TRUE)

hex_bio_df <- hex_bio %>%
  st_drop_geometry()

track_bio_df <- track_bio %>%
  st_drop_geometry()

write.csv(hex_bio_df, "GIS_COVS/hex_bio.csv")
write.csv(track_bio_df, "GIS_COVS/track_bio.csv")
```

Cleaning up space

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget","studyarea",
                          "ss_bath","hex_chem","hex_bio",
                          "track_chem","track_bio","nemobath")))
```

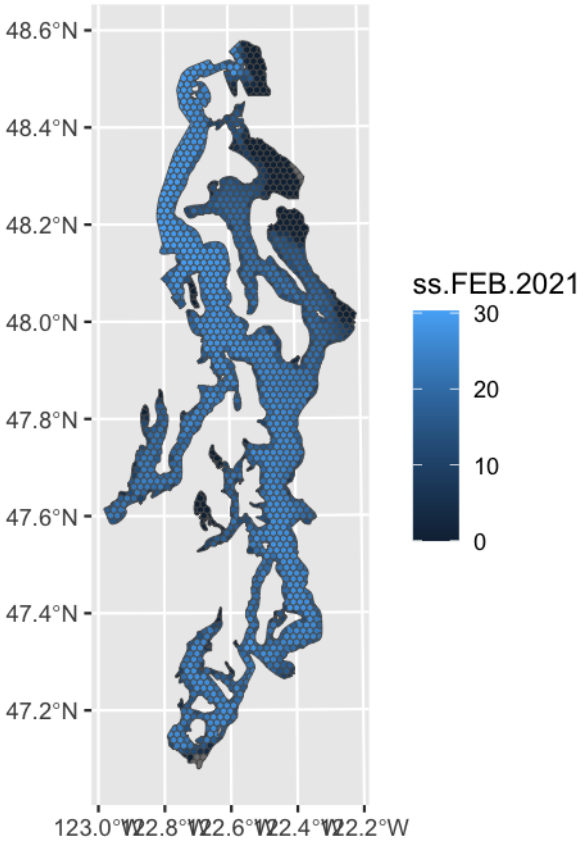
Physics Fields

3d potential temperature, reference salinity, and potential density fields from the SalishSeaCast NEMO model (Soontiens et al, 2016; Soontiens and Allen, 2017) were downloaded from their ERDDAP server on **01/20/2025** from dataset ubcSSg3DPhysicsFields1moV21-11 for all months from 2012 to end of 2024.

- Salinity
- Sea Surface Temperature
- Sigma_theta

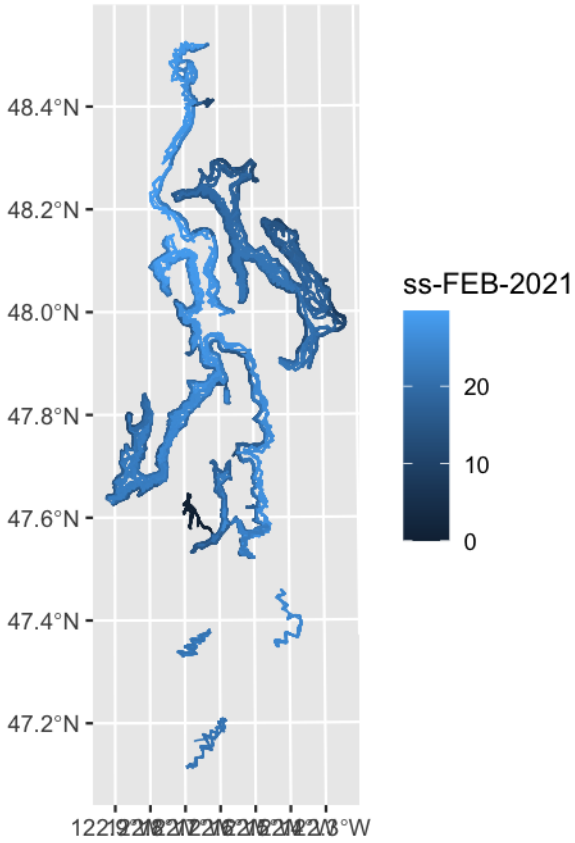
```
folder_path <- "../GIS_COVS/SSC_PHY"

csv_files <- list.files(folder_path, pattern = "\\.csv$", full.names = TRUE)

# Generate simplified names like dfbio1, dfbio2, etc.
simplified_names <- paste0("dfphy", seq_along(csv_files))

# Load all CSV files into a list with simplified names
csv_list <- lapply(csv_files, read.csv)
names(csv_list) <- simplified_names

ss_list<- list()
for (i in 1:length(csv_list)){
  ss_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
    dplyr::select(c(time, gridxy,salinity)) %>%
    mutate(salinity = as.numeric(salinity),
           year = lubridate::year(lubridate::ymd_hms(time)),
           mon = toupper(format(lubridate::ymd_hms(time), "%b")),
           monyear = paste0(mon, "-", year)) %>%
    mutate(salinity = ifelse(is.nan(salinity), NA,salinity)) %>%
    dplyr::select(gridxy,monyear, salinity) %>%
    pivot_wider(names_from = monyear,
                values_from = salinity,
                names_prefix = "ss-")
}

combined_csvss <- bind_rows(ss_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_ss <- vect(combined_csvss, geom = c("longitude", "latitude"),
                  crs = "EPSG:4326")

columns_to_rasterize <- setdiff(names(combined_csvss),
                                c("latitude", "longitude"))

ss_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_ss),
            crs = "EPSG:4326")

  # Rasterize the points for the current column
```

```
  raster_layer <- rasterize(points_ss, r, field = col, fun = "mean", na.rm=T)

  # Add the raster to the list
  ss_raster[[col]] <- raster_layer
}


# Combine all rasters into a stack
ss_stack <- rast(ss_raster)

puget_transformed <- st_transform(puget, crs= crs(ss_stack))

cropped_raster <- crop(ss_stack, puget_transformed)
#mask
masked_raster <- terra::mask(cropped_raster, puget_transformed)


hex_grid_transformed <- st_transform(hex_grids, crs= crs(ss_stack))

ss_ext <- terra::extract(masked_raster,hex_grid_transformed,
                         fun = "mean",weights= T,ID=T,na.rm=T)

hex_ss <- hex_grids %>%
  full_join(ss_ext %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = hex_ss, aes(fill=`ss-FEB-2021`))
```



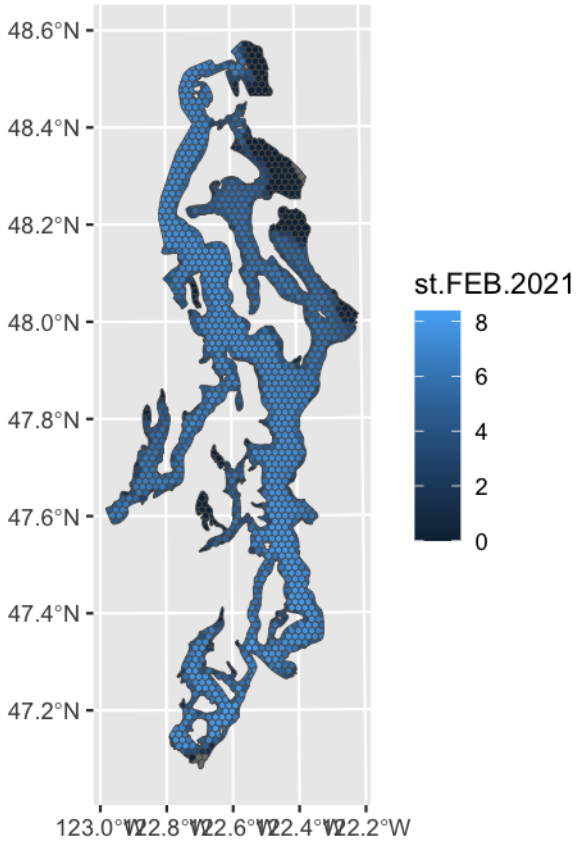
Salinity

```
track_transformed <- st_transform(track_buffer, crs= crs(ss_stack))

ss_ext_track <- terra::extract(masked_raster,track_transformed,
                               fun = "mean",weights= T,ID=T,na.rm=T)

track_ss <- track_buffer %>%
  full_join(ss_ext_track %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = track_ss, aes(fill=`ss-FEB-2021`))
```



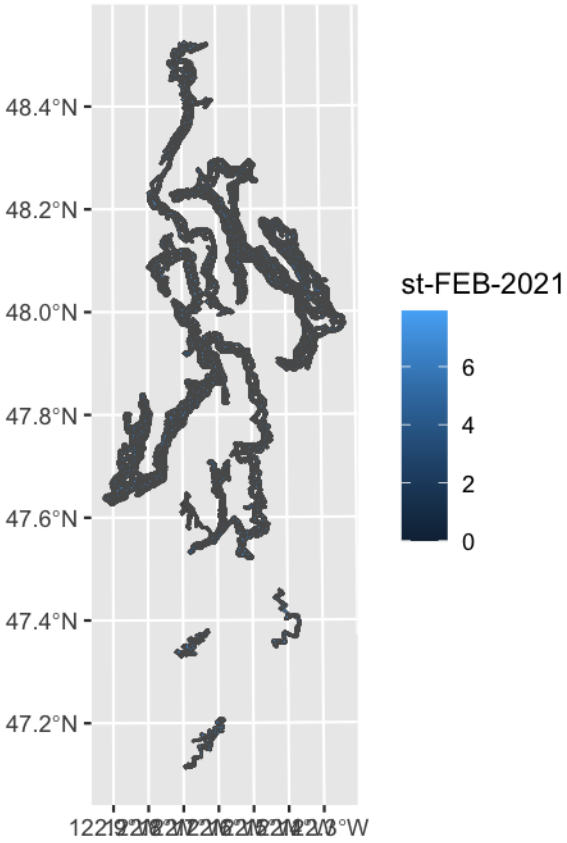

```
st_list<- list()
for (i in 1:length(csv_list)){
  st_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
    dplyr::select(c(time, gridxy,temperature)) %>%
    mutate(temperature = as.numeric(temperature),
           year = lubridate::year(lubridate::ymd_hms(time)),
           mon = toupper(format(lubridate::ymd_hms(time), "%b")),
           monyear = paste0(mon, "-", year)) %>%
    mutate(temperature = ifelse(is.nan(temperature), NA,temperature)) %>%
    dplyr::select(gridxy,monyear, temperature) %>%
    pivot_wider(names_from = monyear,
                values_from = temperature,
                names_prefix = "st-")
}

combined_csvst <- bind_rows(st_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_st <- vect(combined_csvst, geom = c("longitude", "latitude"),
                  crs = "EPSG:4326")
```

```r
columns_to_rasterize <- setdiff(names(combined_csvst),
                                c("latitude", "longitude"))

st_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_st),
            crs = "EPSG:4326")

  # Rasterize the points for the current column
  raster_layer <- rasterize(points_st, r, field = col, fun = "mean", na.rm=T)

  # Add the raster to the list
  st_raster[[col]] <- raster_layer
}


# Combine all rasters into a stack
st_stack <- rast(st_raster)

puget_transformed <- st_transform(puget, crs= crs(st_stack))

cropped_raster <- crop(st_stack, puget_transformed)
#mask
masked_raster <- terra::mask(cropped_raster, puget_transformed)

hex_grid_transformed <- st_transform(hex_grids, crs= crs(st_stack))

st_ext <- terra::extract(masked_raster,hex_grid_transformed,
                         fun = "mean",weights= T,ID=T,na.rm=T)

hex_phy <- hex_ss %>%
  full_join(st_ext %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = hex_phy, aes(fill=`st-FEB-2021`))
```



Temperature

```
track_transformed <- st_transform(track_buffer, crs= crs(st_stack))

st_ext_track <- terra::extract(masked_raster,track_transformed,
                               fun = "mean",weights= T,ID=T,na.rm=T)

track_phy <- track_ss %>%
  full_join(st_ext_track %>%
              rename(numid= ID), by = "numid") %>%
  dplyr::select(-ends_with("converted"))

ggplot()+
  geom_sf(data = track_phy, aes(fill=`st-FEB-2021`))
```



SAVE INTERMEDIATE

```
st_write(hex_phy, "GIS_COVS/hex_phy.gpkg", delete_layer = TRUE)
st_write(track_phy, "GIS_COVS/track_phy.gpkg", delete_layer = TRUE)

hex_phy_df <- hex_phy %>%
  st_drop_geometry()

track_phy_df <- track_phy %>%
  st_drop_geometry()
write.csv(hex_phy_df, "GIS_COVS/hex_phy.csv")
write.csv(track_phy_df, "GIS_COVS/track_phy.csv")
```

Cleaning up space

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy")))
```

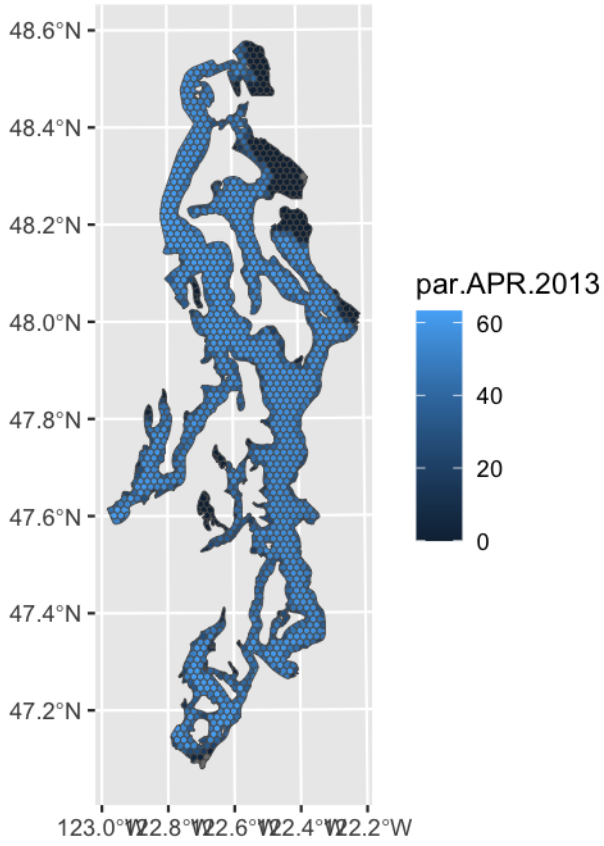
Turbidity Fields

3d photosynthetically available radiation (PAR) and Fraser River turbidity field values from the SalishSeaCast NEMO model (Soontiens et al, 2016; Moore-Maley et al, 2016; Soontiens and Allen, 2017; Olson et al, 2020) were downloaded from their ERDDAP server on **03/05/2025** from dataset ubcSSg3DLightFields1moV21-11.

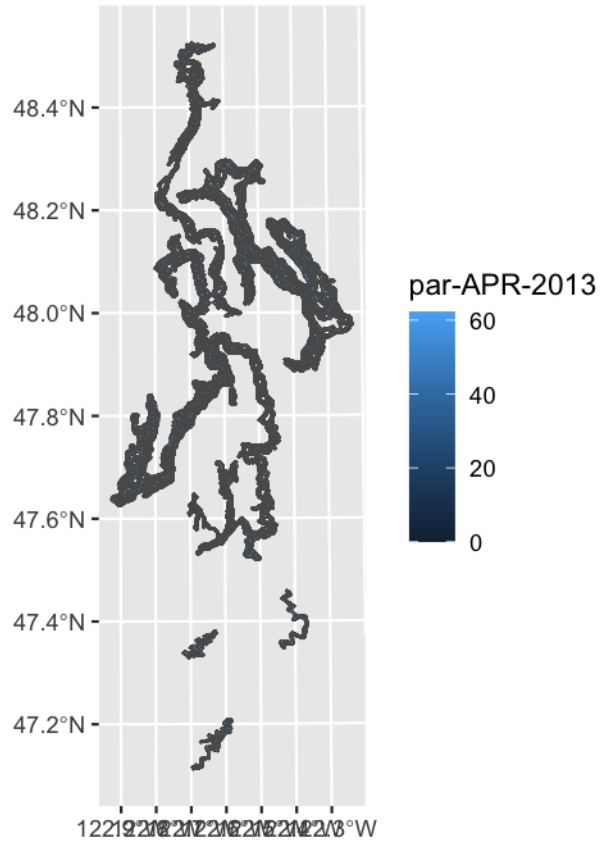
PAR Photosynthetically Available Radiation W m-2

```
folder_path <- "../GIS_COVS/SSC_PAR"

csv_files <- list.files(folder_path, pattern = "\\.csv$", full.names = TRUE)

# Generate simplified names like dfbio1, dfbio2, etc.
simplified_names <- paste0("dflight", seq_along(csv_files))

# Load all CSV files into a list with simplified names
csv_list <- lapply(csv_files, read.csv)
names(csv_list) <- simplified_names

par_list<- list()
for (i in 1:length(csv_list)){
par_list[[i]] <- csv_list[[i]] %>% filter(gridY != "count") %>%
    mutate(gridxy = paste0(gridX, "-",gridY)) %>%
    dplyr::select(c(time, gridxy,PAR)) %>%
    mutate(PAR = as.numeric(PAR),
      year = lubridate::year(lubridate::ymd_hms(time)),
      mon = toupper(format(lubridate::ymd_hms(time), "%b")),
      monyear = paste0(mon, "-", year)) %>%
   mutate(PAR= ifelse(is.nan(PAR),NA,PAR)) %>%
    dplyr::select(gridxy,monyear, PAR) %>%
    pivot_wider(names_from = monyear,
      values_from = PAR,
      names_prefix = "par-")
}

combined_csvpar <- bind_rows(par_list) %>%
  merge(nemobath, by = "gridxy") %>%
  dplyr::select(-c(gridX,gridY, gridxy,bathymetry))


points_par <- vect(combined_csvpar, geom = c("longitude", "latitude"),
                   crs = "EPSG:4326")

columns_to_rasterize <- setdiff(names(combined_csvpar),
                                c("latitude", "longitude"))

par_raster <- list()

# Loop through each column and create a raster layer
for (col in columns_to_rasterize) {
  # Create an empty raster for the extent
  r <- rast(extent = ext(points_par),
    crs = "EPSG:4326")

  # Rasterize the points for the current column
  raster_layer <- rasterize(points_par, r, field = col, fun = "mean", na.rm=T)

  # Add the raster to the list
  par_raster[[col]] <- raster_layer
}
```

```r
# Combine all rasters into a stack
par_stack <- rast(par_raster)


puget_transformed <- st_transform(puget, crs= crs(par_stack))

cropped_raster <- crop(par_stack, puget_transformed)
#mask

masked_raster <- terra::mask(cropped_raster, puget_transformed)

hex_grid_transformed <- st_transform(hex_grids, crs= crs(par_stack))

par_ext <- terra::extract(masked_raster,hex_grid_transformed,
                          fun = "mean",weights= T,ID=T,na.rm=T)

hex_par <- hex_grids %>%
  full_join(par_ext %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = hex_par, aes(fill=`par.APR.2013`))
```



```
track_transformed <- st_transform(track_buffer, crs= crs(par_stack))

par_ext_track <- terra::extract(masked_raster,track_transformed,
                                fun = "mean",weights= T,ID=T,na.rm=T)

track_par <- track_buffer %>%
  full_join(par_ext_track %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = track_par, aes(fill=`par-APR-2013`))
```



SAVE INTERMEDIATE

```
st_write(hex_par, "GIS_COVS/hex_par.gpkg", delete_layer = TRUE)
st_write(track_par, "GIS_COVS/track_par.gpkg", delete_layer = TRUE)

hex_par_df <- hex_par %>%
  st_drop_geometry()

track_par_df <- track_par %>%
  st_drop_geometry()

write.csv(hex_par_df, "GIS_COVS/hex_par.csv")
write.csv(track_par_df, "GIS_COVS/track_par.csv")
```

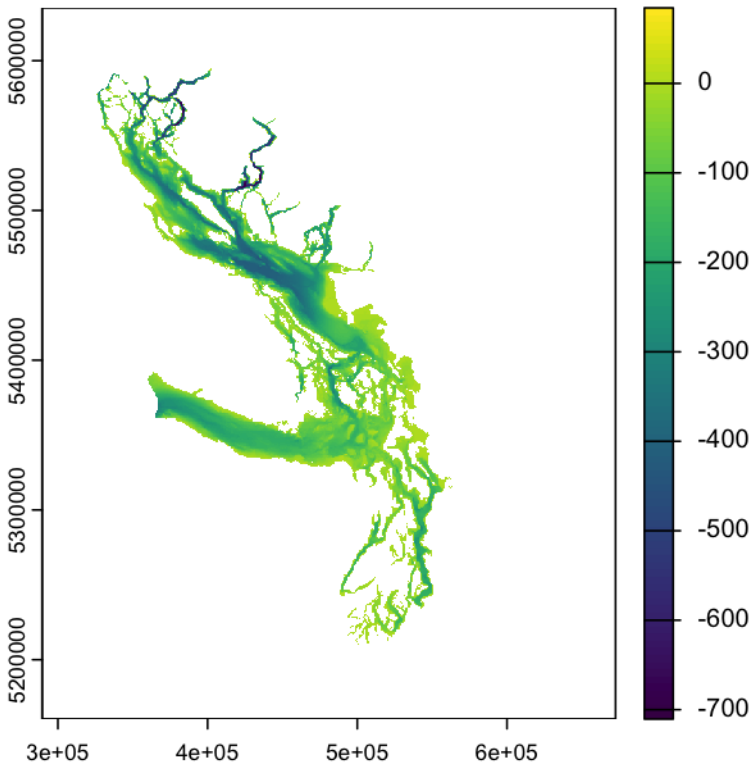
Cleaning up space

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par")))
```

Suitable Habitat

Get Larger Puget Sound Shape

```
plot(ss_bath)
```




```
bath_poly <- st_as_sf(as.polygons(ss_bath, dissolve = TRUE),
                      as_points = FALSE)

# Drop attributes, keeping only geometry
bath_poly_geom <- st_geometry(bath_poly)

# Union all geometries into a single polygon
single_polygon <- st_union(bath_poly_geom)
```

```r
# Convert to sf object
puget_big <- st_sf(geometry = single_polygon)
plot(puget_big)
```

```r
raster_folder <- "../GIS_COVS/MAXENT_2012_2023"

# List all raster files in the folder
raster_files <- list.files(raster_folder, pattern = "\\.(tif|grd|nc)$",
                           full.names = TRUE)

# Import all rasters as a list
raster_list <- rast(raster_files)
plot(raster_list[[5]])
crs(raster_list)

# Assign layer names based on file names (without extension)
names(raster_list) <- tools::file_path_sans_ext(basename(raster_files))
names(raster_list)

## 50km buffer
pbuff <- st_buffer(puget,dist = units::as_units(50000, "m") )

pbuff_T <- st_transform(pbuff, crs=crs(raster_list))

r_clipped <- mask(crop(raster_list, pbuff_T), pbuff_T)
```

```r
puget_t_small <- st_transform(puget_big, crs= crs(r_clipped))

r_NOSEA <- mask(r_clipped, puget_t_small, inverse=T)

#plot(r_NOSEA[[5]])
#res(r_NOSEA[[5]])


# Get layer names
names(r_NOSEA) <- tools::file_path_sans_ext(basename(raster_files))
names(r_NOSEA)

layer_names <- names(r_NOSEA)

## add buffer to center of grid
cents <- st_centroid(hex_grids)

hex_buffered <- st_buffer(cents, dist = units::as_units(25000, "m"))

track_25kmbuffered <- st_buffer(track_buffer,
                                dist = units::as_units(25000, "m"))

#ggplot()+
  #geom_sf(data = puget_buffer)+
 # geom_sf(data = puget, color ="white", fill="white")+
  #geom_sf(data = hex_buffered, fill=NA)


#ggplot()+
  #geom_sf(data = puget_buffer)+
#  geom_sf(data = puget, color ="white", fill="white")+
#  geom_sf(data = track_25kmbuffered, fill=NA)

hex_transformed <- st_transform(hex_buffered, crs= crs(r_NOSEA))

track_transformed <- st_transform(track_25kmbuffered, crs= crs(r_NOSEA))

#ggplot() +
#  tidyterra::geom_spatraster(data = r_NOSEA)+
#  geom_sf(data = track_transformed, fill = NA, color = "white",
# linewidth = 0.5)
```

```r
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par",
                          "r_NOSEA","hex_transformed",
                          "track_transformed")))
```

2012

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[1]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[1]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df1 <- bind_rows(suitability_list)
```

2013

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[2]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)
```

```
   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[2]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df2 <- bind_rows(suitability_list)

beepr::beep(9)
```

2014

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[3]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[3]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_d3 <- bind_rows(suitability_list)

beepr::beep(9)
```

2015

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[4]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[4]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df4 <- bind_rows(suitability_list)

beepr::beep(9)
```

2016

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[5]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
```

```
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[5]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df5 <- bind_rows(suitability_list)

beepr::beep(9)
```

2017

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[6]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[6]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df6 <- bind_rows(suitability_list)
```

```
beepr::beep(9)
```

2018

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[7]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[7]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df7 <- bind_rows(suitability_list)

beepr::beep(9)
```

2019

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[8]], grid_I), grid_I)

  # Extract values
```

```
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[8]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df8 <- bind_rows(suitability_list)

beepr::beep(9)
```

2020

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[9]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[9]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
```

```
}

# Combine all buffers
suitability_df9 <- bind_rows(suitability_list)

beepr::beep(9)
```

2021

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[10]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[10]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df10 <- bind_rows(suitability_list)

beepr::beep(9)
```

2022

Looping through each year separately because it takes an extremely long time to run.

```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)
```

```
   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[11]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
      layer = names(r_NOSEA[[11]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df11 <- bind_rows(suitability_list)

beepr::beep(9)
```

2023

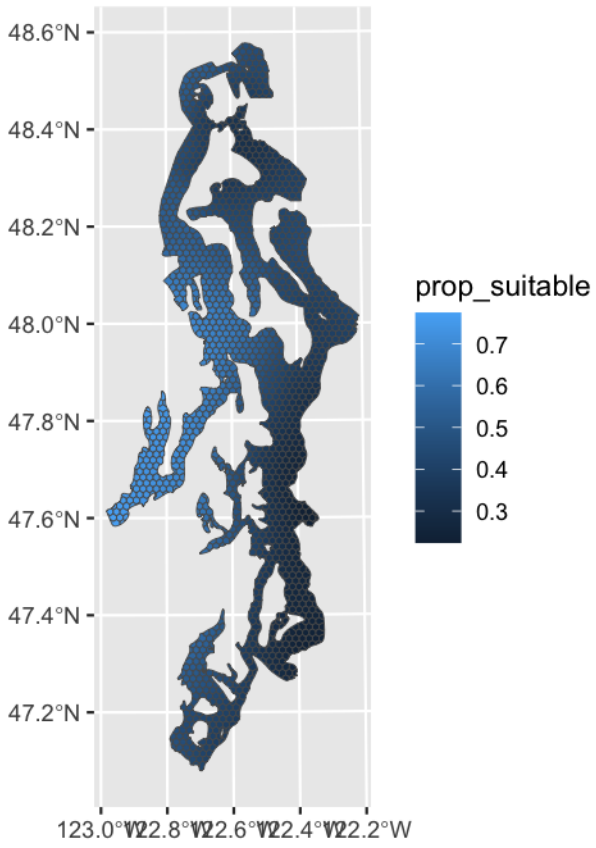
Looping through each year separately because it takes an extremely long time to run.

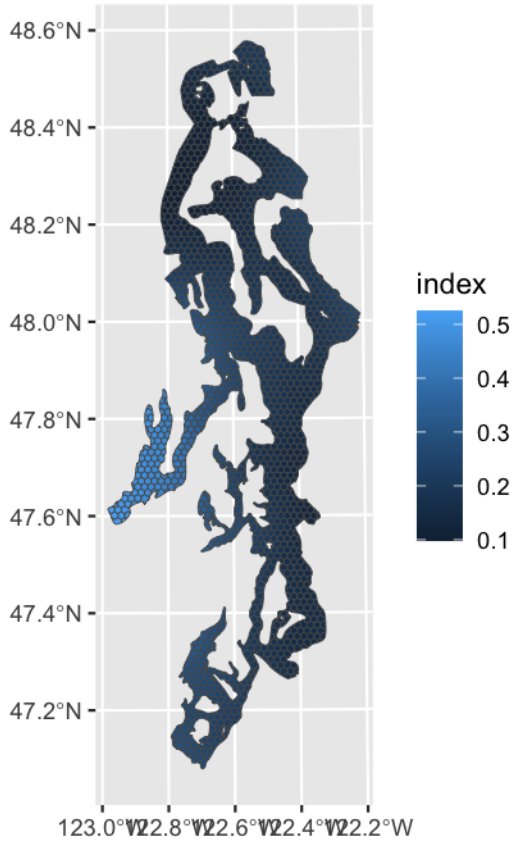
```
suitability_list <- list()

for (i in unique(hex_transformed$ID)) {
  grid_I <- hex_transformed %>% filter(ID == i)

   # Crop and mask just this layer to the buffer
   rl <- mask(crop(r_NOSEA[[12]], grid_I), grid_I)

  # Extract values
  vals <- terra::values(rl)

   total <- length(vals)
   land <- sum(!is.na(vals))
   suitable <- sum(vals > 0.5, na.rm = TRUE)

   prop_land <- land / total
   prop_suitable <- if (land > 0) suitable / land else NA
   index <- prop_land * prop_suitable

    suitability_list[[i]] <- data.frame(
      ID = i,
```

```
      layer = names(r_NOSEA[[12]]),
      prop_land = prop_land,
      prop_suitable = prop_suitable,
      index = index)
}

# Combine all buffers
suitability_df12 <- bind_rows(suitability_list)

beepr::beep(9)
```

Merge together

```
all_suit <- rbind(suitability_df1,
                  suitability_df2,
                  suitability_d3,
                  suitability_df4,
                  suitability_df5,
                  suitability_df6,
                  suitability_df7,
                  suitability_df8,
                  suitability_df9,
                  suitability_df10,
                  suitability_df11,
                  suitability_df12)



hex_suit <- hex_grids %>%
  full_join(all_suit, by = "ID") %>%
  mutate(year = gsub("maxent_hab_mn_","",layer)) %>%
  ## Add 2023 to 2024
 rbind( hex_grids %>%
   full_join(all_suit %>%
               mutate(year = gsub("maxent_hab_mn_","",layer)) %>%
               filter(year == "2023") %>%
               mutate(layer = gsub("2023", "2024", layer),
                      year = "2024"), by = "ID"))

ggplot()+
  geom_sf(data = hex_suit %>%
            filter(year == "2022"), aes(fill=prop_suitable))
ggplot()+
  geom_sf(data = hex_suit %>%
            filter(year == "2022"), aes(fill=index))
```


48.6°N
48.4°N
48.2°N
48.0°N
47.8°N
47.6°N
47.4°N
47.2°N
123.0°W
122.8°W
122.6°W
122.4°W
122.2°W
prop_suitable
0.7
0.6
0.5
0.4
0.3




```
st_write(hex_suit, "GIS_COVS/hex_suit.gpkg", delete_layer = TRUE)

hex_suit_df <- hex_suit %>%
  st_drop_geometry()
write.csv(hex_suit_df, "GIS_COVS/hex_suit.csv")
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed")))
```

Tracklines

2012

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2012")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2012"]], df)
```

```
ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf1 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1")))
```

2013

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2013")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2013"]], df)
```

```
ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf2 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2")))
```

2014

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2014")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2014"]], df)
```

```
ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf3 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3")))
```

2015

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2015")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2015"]], df)
```

```r
ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf4 <- bind_rows(results_list)
```

```r
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4"
                          )))
```

2016

```r
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2016")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2016"]], df)
```

```r
ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf5 <- bind_rows(results_list)
```

```r
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5")))
```

2017

```r
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2017")) %>%
  mutate(ID = row_number())
```

```r
rl <- crop(r_NOSEA[["maxent_hab_mn_2017"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf6 <- bind_rows(results_list)
```

```r
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5","track_resultsdf6")))
```

2018

```r
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2018")) %>%
```

```
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2018"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf7 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5","track_resultsdf6",
                          "track_resultsdf7")))
```

2019

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2019")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2019"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf8 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5","track_resultsdf6",
                          "track_resultsdf7","track_resultsdf8")))
```

2020

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2020")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2020"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf9 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5","track_resultsdf6",
                          "track_resultsdf7","track_resultsdf8",
```

```
                            "track_resultsdf9")))
```

2021

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2021")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2021"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf10 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
```

```
                           "track_resultsdf1","track_resultsdf2",
                           "track_resultsdf3","track_resultsdf4",
                           "track_resultsdf5","track_resultsdf6",
                           "track_resultsdf7","track_resultsdf8",
                           "track_resultsdf9","track_resultsdf10")))
```

2022

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2022")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2022"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf11 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
```

```
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5","track_resultsdf6",
                          "track_resultsdf7","track_resultsdf8",
                          "track_resultsdf9","track_resultsdf10",
                         "track_resultsdf11")))
```

2023

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2023")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2023"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")

  message(paste("Processed chunk", i, "of", length(id_list)))
}
```

```
track_resultsdf12 <- bind_rows(results_list)
```

```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5","track_resultsdf6",
                          "track_resultsdf7","track_resultsdf8",
                          "track_resultsdf9","track_resultsdf10",
                         "track_resultsdf11","track_resultsdf12")))
```

2024

```
df <- track_transformed %>%
  filter(startsWith(as.character(date), "2024")) %>%
  mutate(ID = row_number())

rl <- crop(r_NOSEA[["maxent_hab_mn_2023"]], df)


ids <- unique(df$ID)
chunk_size <- 100
id_list <- split(ids, ceiling(seq_along(ids) / chunk_size))

results_list <- list()

for (i in seq_along(id_list)) {

  chunk_df <- df[df$ID %in% id_list[[i]], ]

  chunk_df <- chunk_df %>%
    mutate(ID2 = row_number())

  ext <- terra::extract(rl, chunk_df)
  colnames(ext)[2] <- "val"

  results_list[[i]] <- ext %>%
    group_by(ID) %>%
    summarise(prop_land = sum(!is.na(val)) / n(),
              prop_suitable = if(sum(!is.na(val)) > 0) sum(val > 0.5,
                                                           na.rm=T) /
                sum(!is.na(val)) else NA,
      index = prop_land * prop_suitable,.groups = "drop") %>%
    rename(ID2 = ID) %>%
    inner_join(chunk_df %>%
                 st_drop_geometry() %>%
                 select(c(ID,ID2, trip_date,date)), by = "ID2")
```

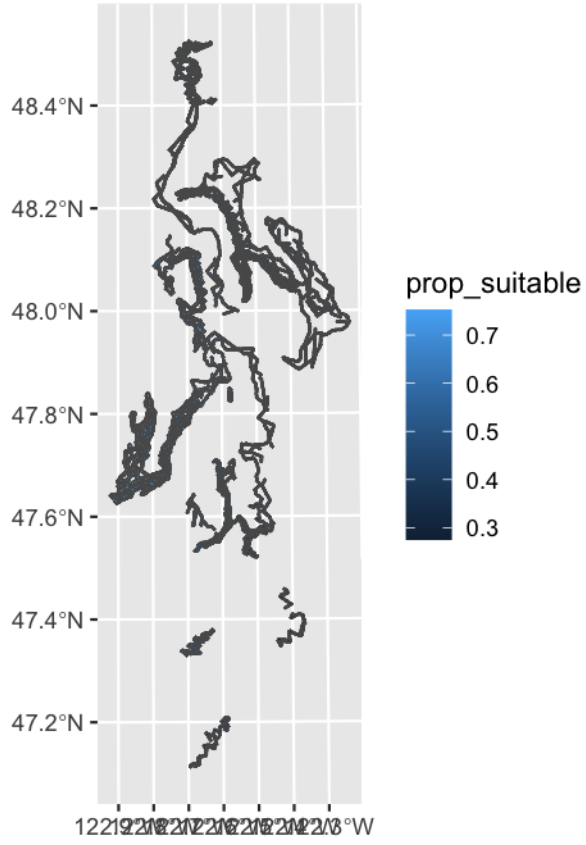
```
  message(paste("Processed chunk", i, "of", length(id_list)))
}


track_resultsdf13 <- bind_rows(results_list)
```

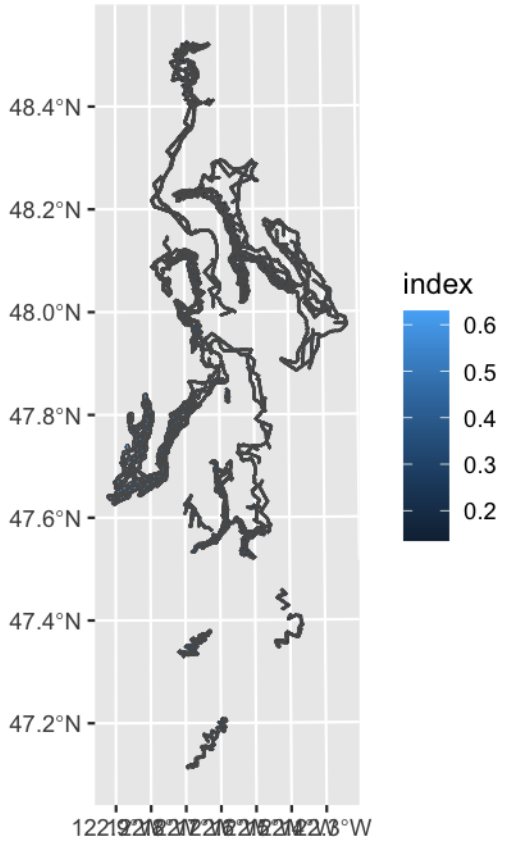
```
rm(list = setdiff(ls(), c("hex_grids","track_buffer","puget",
                          "studyarea", "ss_bath","hex_chem",
                          "hex_bio", "nemobath","hex_phy",
                          "track_chem","track_bio","track_phy",
                          "hex_par","track_par","hex_suit",
                          "r_NOSEA","track_transformed",
                          "track_resultsdf1","track_resultsdf2",
                          "track_resultsdf3","track_resultsdf4",
                          "track_resultsdf5","track_resultsdf6",
                          "track_resultsdf7","track_resultsdf8",
                          "track_resultsdf9","track_resultsdf10",
                         "track_resultsdf11","track_resultsdf12",
                         "track_resultsdf13")))
```

Merge together

```
all_suit <- rbind(track_resultsdf1,
                  track_resultsdf2,
                  track_resultsdf3,
                  track_resultsdf4,
                  track_resultsdf5,
                  track_resultsdf6,
                  track_resultsdf7,
                  track_resultsdf8,
                  track_resultsdf9,
                  track_resultsdf10,
                  track_resultsdf11,
                  track_resultsdf12,
                  track_resultsdf13) %>%
  select(-c(ID,ID2)) %>%
  group_by(trip_date, date) %>%
  summarise(prop_land = mean(prop_land),
            prop_suitable = mean(prop_suitable),
            index = mean(index), .groups = "drop") %>%
  ungroup()


track_suit <- track_buffer %>%
  group_by(trip_date, date) %>%
  summarise(total_area = as.numeric(sum(st_area(geom)))/1e6,
            geom = st_union(geom), .groups = "drop") %>%
  full_join(all_suit, by = c("trip_date","date")) %>%
  ungroup() %>%
  mutate(year = str_sub(date, 1,4))
```

```r
ggplot()+
  geom_sf(data = track_suit %>%
            filter(year == "2022"), aes(fill=prop_suitable), color = NA)
ggplot()+
  geom_sf(data = track_suit %>%
            filter(year == "2022"), aes(fill=index))


ggplot()+
  geom_sf(data = track_suit %>%
            filter(trip_date == "Z1S3P25Z-ZR1 2012-05-24"),
          aes(fill=prop_suitable))
```






```
st_write(track_suit, "GIS_COVS/track_suit.gpkg", delete_layer = TRUE)

track_suit_df <- track_suit %>%
  st_drop_geometry()

write.csv(track_suit_df,"GIS_COVS/track_suit.csv")
```

Save

```
save(hex_chem,hex_bio,hex_suit,
     hex_phy,hex_par,
     track_bio,track_chem,
     track_par,track_phy, track_st,
     track_suit, nemobath,file = "GIS_COVS/Dynamicgrid_data.RData")

save(hex_chem_df,hex_bio_df,hex_suit_df,
     hex_phy_df,hex_par_df,
     track_bio_df,track_chem_df,
     track_par_df,track_phy_df,
     track_suit, nemobath,file = "GIS_COVS/Dynamicgrid_DF.RData")
```